



[</> Documentation](#) [📁 Categories](#) [🏷️ Tags](#) [✎ Contribuer à Yoda](#)

Shell TIPS

Shell Tips

Les bases

Lire un fichier

- `cat` : affiche l'ensemble du fichier dans le terminal
- `tail` : affiche les 10 dernières lignes
- `head` : affiche les 10 premières lignes

Alt text

La base: découvrir comment fonctionne une commande

- `man` : documentation offline des commandes (/!\ souvent les outils historiques Linux ont un `man`, les outils plus récents n'en ont généralement pas)
 - `man man` pour savoir comment lire un `man`

Alt text

- `tldr` : alternative "plus moderne" au `man`
 - <https://github.com/tldr-pages/tldr>
- `cheat.sh` : autre alternative
 - Embarque `tldr`
 - <https://github.com/chubin/cheat.sh>

String et quotes

- Quote simple: rien n'est interprété

Alt text

- Double quote

Alt text

- Dollar single quote (ANSI-C quote): Les caractères échappés sont interprétés (tab, quote, newline, etc...)

Alt text

Navigation dans le terminal

- `cd -` : aller dans le dossier précédent
- `dirs` : Affiche la pile de répertoires.
- `cd ~2` : aller dans le dossier 2 de la stack

Tips : `alias dirs='dirs -v'` dans votre `bashrc` ou `zshrc`

Alt text

- `hash -d D=DOSSIER` :
 - `cd ~D` pour y accéder

Alt text

Historique de commandes

- `ctrl+r` : Recherche dans l'historique
- `ctrl+r` : Suivant
- `ctrl+s` : Précédent

Alt text

- `fzf-history` : Recherche dans l'historique via FZF
 - <https://github.com/junegunn/fzf>
- `mcfly` : Autre outil de gestion de l'historique
 - <https://github.com/cantino/mcfly>

Historique de commandes – effacer l'historique

- `_cmd` (Il y a un espace devant la commande)
 - Executer une commande sans l'ajouter dans l'historique
 - `setopt | grep histignoreospace` (zsh)
 - Variable d'environnement `HISTCONTROL=ignoreospace` / `HISTCONTROL=ignoreboth` (bash)
- `ctrl+u` : Efface/Cut la ligne courante (utile pour les mots de passe)

- `ctrl+y` : Colle le morceau de texte cut le plus récent

Alt text

Historique de commandes - expansion

- `!!` : exécute la dernière commande
 - `sudo !!` : exécute la dernière commande en sudo

Alt text

- `!x` : exécute la dernière commande qui commence par `x`
 - `!ssh` : exécute la dernière commande qui commence par `ssh`

Alt text

- `fc` : ouvre la dernière commande dans l'éditeur par défaut pour la corriger plus facilement

Un peu plus technique :

- `!:*` : désigne tous les arguments de la commande précédente
- `!:$` : désigne le dernier argument de la commande précédente
- `!:2` : désigne le deuxième argument de la commande précédente
- `!#:N` : réutilise le Nième mot de la commande courante

Alt text

- `^str1^str2` : substitue la première occurrence de `str1` par `str2` au sein de la dernière commande

Alt text

Un peu plus technique :

- `!!:gs/pattern/replace` :
 - `!!` : dernière commande
 - `g` : toutes les occurrences
 - `s` : mode replace (substitute)

Alt text

Screen

- `ctrl+l` : clear the screen except the prompt

- `ctrl+s` : stop all output to the screen (utile pour des commandes trop verbeuses)
 - Peut mettre en pause le processus en lui-même s'il a besoin de STDOUT dans son fonctionnement (ex: ffuf)
- `ctrl+q` : resume output to the screen

Alt text

Navigation dans la ligne de commande

- `ctrl+a` : aller au début de la ligne
- `ctrl+e` : aller en fin de ligne
- `ctrl+u` : supprimer toute la ligne
- `ctrl+w` : supprimer le mot avant le cursor
- `ctrl+k` : supprimer la ligne après le cursor

! Ces raccourcis sont dépendant du shell/framework utilisé (zsh, bash, oh-my-zsh, etc...)

Sinon dans iTerm2: Natural Text Editing

- `cmd+[←]` : aller au début de la ligne
- `cmd+[→]` : aller en fin de ligne
- `cmd+[DEL]` : supprimer toute la ligne
- `option+[←]` : se déplacer vers la gauche de mot en mot
- `option+[→]` : se déplacer vers la droite de mot en mot

Alt text

Alias

- Alias simple: `alias cmd="long command with arguments"`
 - Ex: `alias ll="ls -lah"`
- Alias global: `alias -g cmd="command"`
 - Ex: `alias -g ...="../.." => ls ... = ls ../..`
 - Permet de substituer n'importe où dans la commande
- Alias suffixe: `alias -s extension="commande"`

- `alias -s jq="jq ." => ./test.json => JSON bien formaté`
- `alias -s md=bat => ./index.md => affiche le fichier MD au sein de bat`
- `command cmd` : permet d'appeler la command (dans `$PATH`) même si un alias/function l'override \cmd: idem
- `=cmd` : «alias» du chemin absolu vers le binaire dans le path (`ls -lah =ssh` équivaut à `ls -lah /usr/bin/ssh`)

Alt text

Gestion des processus

- `ctrl+c` : interrupt (kill) process (SIGINT)
- `ctrl+z` : suspend process (SIGTSP)
 - `fg` pour reprendre l'exécution au premier plan (foreground)
 - `bg` pour reprendre l'exécution en arrière-plan (background) (équivalent de `command &`)
- `ctrl+d` : send EOF marker ~exit

Alt text

- `&`, `nohup` et `disown`
 - exécutent une commande en arrière-plan avec certaines différences :
 - `&` : STDOUT et STDERR sont reliés au terminal depuis lequel la commande a été exécutée. Si le terminal parent est fermé, la commande exécutée en arrière-plan va s'interrompre
 - `nohup` : Les sorties STDOUT et STDERR seront redirigées dans un fichier `nohup.out` par défaut. Même si le terminal est fermé, la commande continuera de s'exécuter
 - `disown` : Permet de détacher du terminal une commande qui n'a pas été exécutée en arrière-plan. Les sorties STDOUT et STDERR sont toujours reliées au terminal.

```
$ nmap xmco.fr
$ ctrl+Z
$ bg
$ disown
```



BONUS: `screen` / `tmux` : utilitaires qui permettent de créer des sessions de terminal persistantes et détachables

Text & Files processing

Text processing - grep

- grep :
 - -A X (show X line **A**fter pattern)
 - -B X (show X line **B**efore pattern)
 - -C X (show X line of **C**ontext (after/before)) Alt text
 - -w : mot entier (séparé par des caractères n'étant pas des caractères constituant des mots)
 - -i : recherche insensible à la casse
 - -r : recherche récursivement dans le dossier
 - -v : inverse la recherche (lignes ne contenant pas le pattern)
 - -o : n'affiche que les caractères correspondant au pattern recherche (et pas toute la ligne)
 - -l : affiche la liste des fichiers contenant le pattern recherché (petit L)
 - -I : ne cherche pas dans les fichiers binaires (grand i)
- ripgrep : (alternative plus rapide)
- Comment grep quelque chose commençant par -BLABLA ?

Alt text

```
grep -- -
```



Alt text

Explication : le « -- » indique au programme (ici grep) qu'il ne faut pas considérer le reste de la ligne comme des flags

Text processing - awk

- awk '/PATTERN/' '{ACTION}' – manipulation de colonnes
 - Recherche dans un ou plusieurs fichiers la donnée correspondant au pattern.
 - À chaque pattern trouvé peut être associée une action
 - \$1 , \$2 spécifie les numéros des champs par rapport à un séparateur, où \$0 désigne la ligne entière

- Afficher certains champs d'un fichier contenant un séparateur : `awk -F ' [SEPARATOR] ' '{print $N}' file.txt` : où N est le numéro du champ à afficher

Alt text

- `awk '/pattern/{print $N}' file.txt`

Alt text

- Afficher les mots de passe dans un pot john qui font plus de 4 caractères et moins de 6 caractères :

```
awk -F ':' 'length($2) < 6 && length($2) > 4 {print $2}' john.pot
```

```
awk -F [SEPARATOR] '[CONDITION1] && [CONDITION2] {ACTION}' fichier
```

- Afficher un bloc de texte entre deux patterns :
 - `awk '/start pattern/,/end pattern/' file.txt`
 - `awk '/SELECT/,/FROM/' file.txt`

Alt text

- Parser 2 champs sur une ligne différente avec le même séparateur:
 - `awk -F '=' '/pattern1/{VAR1=$2} /pattern2/{print VAR1="$2}'`

Alt text

```
cat test.json | tr -d '"' ',' | awk -F ":" '/user/{USER=$2} /pass/{PASS=$2} /hostname'
```

Alt text

- Dans quel cas est-il préférable d'utiliser `awk -F` à `cut -d` ?

Alt text

=> Awk ne tient pas compte des occurrences du séparateur

Sort & Uniq

- `sort` : trie un fichier
 - Peut prendre des formes de tris complexes: trier des IP:

- `sort -n -t . -k 1,1 -k 2,2 -k 3,3 -k 4,4`
- `sort -u` : trie et supprime les doublons
- `sort -c` : trie et affiche le nombre d'occurrences dans un fichier
- `sort -n` : trie en utilisant les valeurs numériques
- `uniq` : manipulation sur les doublons dans un fichier (/!\ le fichier doit être trié)
 - `uniq` : n'affiche les lignes qu'une seule fois (supprime les doublons)
 - `uniq -u` : n'affiche que les lignes n'apparaissant qu'une seule fois
 - `uniq -d` : n'affiche que les lignes apparaissant plusieurs fois
 - `uniq -c` : affiche le nombre d'occurrences pour chaque ligne
 - Trier un fichier par nombre d'occurrences des lignes: `cat file | sort | uniq -c | sort -nr`
- Petit bonus: supprimer les doublons sans trier:
 - `awk '!x[$0]++'`

Column

- Formate une sortie sous la forme de colonnes :
 - `column -s(separator) -t`
 - `column -s: -t /etc/passwd`

Utile pour les fichiers CSV par exemple (ou les dumps de tables SQLMAP)

Alt text

Alt text

Sed

- Permet de transformer ou filtrer un flux de texte
 - Syntaxe : `sed 'PATTERN/REGEXP/REPLACEMENT/FLAGS' filename`
Exemple de FLAGS:
 - `p` : pour print
 - `d` : pour delete
 - `i` : case insensitive (check sur BSD)
 - `g` : remplace toutes les occurrences de la ligne

- On peut remplacer « / » par n'importe quel caractère
 - `sed 's/REGEX/T0REPLACE/' file`
 - `sed 's+REGEX+T0REPLACE+g' file`
- Afficher les lignes 3 à 5 d'un fichier
 - `sed -n '3,5p' /etc/passwd`
- Supprimer un bloc de texte entre deux patterns:
 - `sed '/start pattern/,/end pattern/d' file.txt`

Tee

- Afficher sur stdout et dans un fichier
 - `[command] | tee file.log`
 - `./testssl.sh https://www.xmco.fr | tee TESTSSL-www.xmco.fr.log`
- Modifier un fichier nécessitant des permissions sudo
 - `echo "test " | sudo tee -a fichier`

Alt text

JQ

- Afficher le JSON correctement formaté
 - `cat file.json | jq '.'`
- Accéder à un champ
 - `cat file.json | jq '.name'`
- Accéder aux champs au sein des sous-objets
 - `cat file.json | jq '.name.object'`
- Accéder aux objets au sein d'un tableau
 - `cat file.json | jq '.table[]'`
- Pouvoir "grepper" un JSON
 - <https://github.com/tomnomnom/gron>
 - `cat file.json | jq '[leaf_paths as $path | {"key": $path | join("."), "value": getpath($path)}] | from_entries'`

```
[
  {
    "chars": 0,
    "code": 302,
    "payload": "",
    "lines": 0,
    "location": "lms/",
    "method": "GET",
    "post_data": [],
    "server": "cloudflare",
    "url": "https://domain.com/",
    "words": 0
  },
  {
    "chars": 0,
    "code": 200,
    "payload": "",
    "lines": 0,
    "location": "lms/",
    "method": "GET",
    "post_data": [],
    "server": "cloudflare",
    "url": "https://domain.com/",
    "words": 0
  },
]
```

- `cat file.json | jq ".[] | select(.code | contains(\"302\") | not )"`

Alt text

```
jq -r '.[] | "
  \ (keys[] as $k | select( $k | startswith("proto"))) | .[$k])
  ://\(.user)
  :\(.pass)
  @\(.hostname)
  :\(.port)
  "'
```

Alt text

Parcourir les lignes d'un fichier - quizz

Quizz 1

```
$ cat test.txt
1 2
34
5
$ for i in "$(cat test)";do echo "$i:$i";done
```

A	B	C	D	E
Rien	1 2:1 2	1 2	1:1	1 2345
	34:34	34	2:2	
	5:5	5:1 2	34:34	
		34	5:5	
		5		

```
1 2
34
5:1 2
34
5
```

Un seul élément est passé au for (le cat est coté, donc l'ensemble du contenu du fichier est passé en tant qu'un seul élément)

Quizz 2

```
$ cat test.txt
1 2
34
5
$ for i in $(cat test);do echo "$i:$i";done
```

A	B	C	D	E
Rien	1 2:1 2	1 2	1:1	1 2345
	34:34	34	2:2	
	5:5	5:1 2	34:34	
		34	5:5	
		5		

```
1:1
2:2
34:34
5:5
```



Le cat n'est pas quoté, donc le fichier est splitté selon l'IFS (donc sur les newline, espaces et tabulations)

For ou While

Boucle `for` ou Boucle `while` ?

- boucle `for` -> on connaît le nombre d'éléments à parcourir
- boucle `while` -> on lit tant qu'il y a des lignes à lire

Deux problèmes à gérer :

- Ma ligne contient des espaces
- Ma ligne contient des caractères spéciaux ou des caractères d'échappement

While

For

```
while read -r line
do
// Traitement
done < file
```

```
IFS="\n"
for line in $(cat file)
do
//Traitement
done
```

- L'option `-r` indique à la commande `read` de ne pas interpréter les caractères d'échappement
- `IFS` : (Internal Field Separator) pour définir le caractère de séparation des champs à

utiliser dans la boucle for.

- Par défaut `IFS=$' \t\n'` : les espaces, sauts de lignes et tabulation sont les séparateurs

While & Pipe

- `cat file | while (1)`

```
$ cat test .txt
1 2
34
5

$ cat script1.sh
#!/bin/bash
data=""
cat test.txt | while read -r line;do
    data+="$line"
done
echo "$data"

$ ./script1.sh
```

- `while < file (2)`

```
$ cat test .txt
1 2
34
5

$ cat script2.sh
#!/bin/bash
data=""
while read -r line;do
    data+="$line"
done < test.txt
echo "$data"

$ ./script2.sh
```

A	B	C	D	E
Rien	1 2:1 2	1 2	1:1	1 2345
	34:34	34	2:2	
	5:5	5:1 2	34:34	
		34	5:5	
		5		

Réponse:

- 1: A (Rien) Le while s'effectue dans un sous-shell (en raison du pipe), ainsi il n'a pas accès à la variable du shell parent `data`, mais à une variable locale.
Le sous-shell est quitté en sortie de while, ainsi, le `echo "$data"` affiche la variable du shell parent, qui n'a pas été modifiée.
- 2: E (1 2345) Aucun sous-shell n'est utilisé (le fichier est passé via une redirection).
Ainsi, la variable `data` définie en début de script est la même que dans le while.

Xargs

Utile pour appliquer la même commande sur une liste de fichiers

- Prend une chaîne séparée par des espaces et la convertie en une liste d'arguments
 - `echo "file1 file2 file3" | xargs cat -> cat file1 file2 file3`
- Parallélisation type worker/queue avec un nombre de job défini:
 - `xargs -P $nb_jobs`

Files & directories

- Appliquer un traitement sur plusieurs fichiers avec find
 - Éviter :
 - `find . -name "*.c" -exec ls -al {} \;`
 - Exécute la commande `ls` sur chaque fichier
 - Préférer :
 - `find . -name "*.c" | xargs ls -al`
 - Construit une liste d'arguments à partir de la sortie produite par `find` et passe cette liste à `ls`

Alt text

Find

Alternative à find : `fd ( brew install fd )`

- `fd [options] [pattern] [path]`
 - `-e [extension]`
 - `fd -e jpg png svg`
 - `-E [pattern] (exclude)`
 - `-g [globing]`
 - Plus besoin de `/dev/null`
 - Plus rapide que find
 - `-t` : type (f file, d directory, etc.)
 - `-d` : max-depth
 - Par défaut, ne cherche pas les fichiers / répertoires cachés (`-H`)
 - Par défaut, case insensitive (sauf si le pattern contient une lettre majuscule (smart-case))

Files and directories

- Easy-to-read recursive file listing
 - `find . -ls`

Alt text

- `tree`
- `tree -d` : liste uniquement les directory
- `tree -L [N]` : N désigne le niveau de profondeur de l'arbre

Alt text

Comm

- `comm` : affiche les lignes en commun ou non (les fichiers doivent être triés)
 - `comm -1 -2 file1 file2` : les lignes en commun
 - `comm -2 -3 file1 file2` : les lignes uniquement dans file1
 - `comm -1 -3 file1 file2` : les lignes uniquement dans file2
- Lister les fichiers les plus récents
 - `ls -t` (penser à t pour time)
 - Si vous utilisez exa (`-s new`)

Paste

- `paste` permet de combiner 2 fichiers lignes par ligne
 - `paste -d ':' fichier1 fichier2` va générer

```
ligne1_fichier1:ligne1_fichier2
ligne2_fichier1:ligne2_fichier2
ligne3_fichier1:ligne3_fichier2
ligne4_fichier1:ligne4_fichier2
```



Utile pour générer des couples identifiant:mdp à partir de deux fichiers différents.

Misc

- `<([COMMAND])` : traite la sortie de la commande comme si c'était un fichier (redirige la sortie de la commande vers un fd)
- `join`
 - Rassemble les lignes de deux fichiers ayant un champ commun (les fichiers doivent être triés)

Alt text

- `{start..end..step}`
 - `echo {1..10..2}`
 - `echo {00..10..2}`
- Que va faire cette commande ?

Alt text

Réponse: Elle va vider le fichier sans rien faire.

En effet, lors du lancement de la commande, le shell va tronquer le fichier, avant d'essayer de le lire via `cat`.

- Comment ré-écraser le contenu d'un fichier qu'on utilise en tant que stdin ?
 - `sponge` (brew install sponge)

Alt text

Permet d'éviter de créer plein de fichiers temporaires

/!\ ATTENTION À L'ÉCRASEMENT DE FICHIERS INVOLONTAIRES

Less

- `less -R` (ou `less -r`) : afficher les couleurs dans un fichier (rendu des codes ANSI)
 - `F` : équivalent de `tail -f` permet de lire un fichier tout en voyant les modifications

Alt text	Alt text
----------	----------

Split

- `split` : permet de découper un gros fichier en plusieurs fichiers
 - peut aider à manipuler de gros fichiers en les divisant (transfert, traitement, analyse ou autre)

`split -l 1000 mongrosfichier` : découpe mongrosfichier en fichiers de 1000 lignes.

- Récupérer le nom du fichier à partir du chemin:
 - `basename "/path/to/binary" => binary`
- Récupérer le chemin vers le dossier contenant un fichier
 - `dirname "/path/to/binary" => /path/to`

Misc

- Modifier un fichier via le réseau
 - `vim scp://remote-user@remote-host//path/to/file`

"AdminSys"

SSH

- Copier une clé SSH sur un hôte distant au sein du fichier `authorized_keys` (en ayant eu préalable un login:mdp)
 - `ssh-copy-id -i id_rsa remote-host`
- Utiliser une clé SSH spécifique pour un hôte distant
 - Créer une entrée dans `~/.ssh/config`

Host pentest



```
Hostname 51.159.66.249
User [USER]
Port 42007
```

```
$ ssh pentest
```



Exemple de config ssh:

```
Host pentest socks_pentest agile-buster external_socks
  HostName pentest-echo.xmco.tech
  User [USER]
  Port 42007

Host socks_pentest-echo
  DynamicForward 8000

Host external_socks_pentest-echo
  Localforward 2222 127.0.0.1:2222

Host agile-buster
  LocalForward 9000 127.0.0.1:5000
  LocalForward 9005 127.0.0.1:3000

Host parpin
  Hostname scanly.xmco.fr
  User anything
  DynamicForward 8000
  StrictHostKeyChecking no
  UserKnownHostsFile /dev/null
  LogLevel DEBUG
```



- Kill un SSH qui a freeze : <Enter>~.
- Help : <Enter>~?
- Ouvrir un port : <Enter>-C -L22 :localhost :22

Note: pour interagir avec des sessions SSH imbriquées, répéter ~

~?

Supported escape sequences:

- ~B - send a BREAK to the remote system
 - ~C - open a command line
 - ~R - request rekey
 - ~. - terminate connection (and any multiplexed sessions)
 - ~V/v - decrease/increase verbosity (LogLevel)
 - ~^Z - suspend ssh
 - ~# - list forwarded connections
 - ~& - background ssh (when waiting for connections to terminate)
 - ~? - this message
 - ~~ - send the escape character by typing it twice
- (Note that escapes are only recognized immediately after newline.)

Shell scripting

- Shellcheck: outil d'analyse des scripts bash
 - <https://github.com/koalaman/shellcheck>
- Redirections & substitution de processus
 - `cmd > file` Redirige la sortie standard de la commande `cmd` vers le fichier `file` (écrase `file`)
 - `cmd >> file` Redirige la sortie standard de la commande `cmd` vers le fichier `file` (ajoute à la fin de `file`)
 - `cmd1 <(cmd2)` Redirige STDOUT de `cmd2` vers un file descriptor manipulable par `cmd1`
 - `cmd1 >(cmd2)` expose STDIN de `cmd1` comme file descriptor pouvant être utilisé par `cmd2`
 - `cmd < FICHIER` Mets `FICHIER` en stdin de `cmd` (~ `cat FICHIER | cmd`)
 - `cmd <<EOF` : Heredoc : met le texte en entrée de la commande (multiline)
 - `cmd <<< "string"` met le texte « `string` » en stdin de `cmd`
 - `cd XXX; cmd; cd .. =>` Utilisation de sous-shell:
 - `(cd XXX; cmd)`

Bash substitution

- Variable par défaut
 - `VAR="${VARIABLE:-default}"`
- Substring:

- VAR="\${VARIABLE#PREFIX}" : Supprime le préfix (plus petite correspondance)
- VAR="\${VARIABLE##PREFIX}" Supprime le préfix (plus grande correspondance)
 - Ex: basename "\$VARIABLE" correspond à "\${VARIABLE##*/}"
- VAR="\${VARIABLE%PREFIX}" Supprime le suffix (plus petite correspondance)
 - f="image.jpg.bak" echo "\${f%.*}" => image.jpg
- VAR="\${VARIABLE%%PREFIX}" Supprime le suffix (plus grande correspondance)
 - f="image.jpg.bak" echo "\${f%%.*}" => image

Les tableaux

- Définir un tableau
 - array=("elem1" "elem2" "elem3")
 - Expansion d'un tableau dans une string:
 - string="\${array[@]}"
 - Ajouter un élément dans un tableau:
 - array+=("elem4")
 - Taille d'un tableau:
 - size="\${#array[@]}"
- Concaténer deux tableaux:
 - array("\${array1[@]}" "\${array2[@]}")
- Itérer sur un tableau:

```
for elem in "${array[@]};do
  // traitement
done
```



Misc

- Savoir si on est sur un système 32bits ou 64bits
 - getconf LONG_BIT
- Avoir le PID d'un programme en cours d'exécution
 - pgrep firefox
- cmd | pbcopy : met la stdout de cmd dans le presse-papier
- pbpaste : affiche le contenu du presse-papier

- `compdef _precommand proxychains` : permet d'avoir l'autocomplétion "standard" des commandes après `proxychains`
- Convertir un fichier texte depuis un format windows vers un format unix et vice-versa
 - `dos2unix file.txt`
 - `unix2dos file.txt`
- `tar` : moyen mnémotechnique
 - **Compresser**: `tar czf archive.tar.gz target` (**C**ompress **Z**e **F**ile)
 - **Extraire**: `tar xzf archive.tar.gz` (**X**tract **Z**e **F**ile)
- Afficher l'utilisation du disque en interactif
 - `ncdu`
 - `/!\ -x` pour qu'il ne traverse pas les points de montage (sinon il risque de crawler tout le xéon...)

Alt text

- `htop` : top interactif

Alt text

- `cal` : calendrier en CLI

Alt text

Globing & expansion

- Glob `~=` regex simplifiée
 - `?` : match un caractère
 - `*` : match tout (dans un dossier donné)
 - `**` : match tout y compris dans les sous dossier
 - `[]` : ou
 - `[:upper:]` : majuscules
 - `[:lower:]` : minuscules
 - `[:digit:]` / `[0-9]` : chiffres

Alt text

- Brace Expansion: créer plusieurs string (séparées d'un espace) à partir d'une string contenant `{}` :

- `ls /etc/{shadow,passwd} => ls /etc/shadow /etc/passwd`

Alt text

Alt text

Misc

- Quand le terminal part en cacahuète (coupure SSH par exemple):
 - `reset`

Alt text

- Faire une copie d'un fichier :
 - `cp FILE{,.BAK}`
 - `cp FILE{OLD,NEW}`
 - `cp {,NEW-}FILE`
- Renommer le fichier `script1.sh` en `script2.sh` avec cette méthode ?
 - `cp script{1,2}.sh`
- Backup d'un fichier: `cp file{,.bak}`
- Changer l'extension d'un fichier: `mv file{.old,.new}`
- Créer un backup d'un fichier par date: `cp test{,``date +%F``}`

Bash Scripting

- `set -ueo pipefail`
 - `-u` : Quitte si utilisation d'une variable non initialisée
 - `-e` : Arrêt si commande fail (Ajouter `|| true` pour annuler localement si besoin. ex: `sudo crontab -u myuser -l` met une erreur si vide -> `sudo crontab -u myuser -l || true`)
 - `-o pipefail` : propage l'erreur dans un pipe : se combine bien avec `-e` (Seule la dernière commande du pipe est traitée par `-e`)
 - `-x` : Debug: affiche chaque commande avant son exécution
- Toujours quotter les variables (`"$VAR"` plutôt que `$VAR`)
 - Si besoin d'une expansion, utiliser une liste et l'appeler via `"${VAR[@]}"`
- Utiliser `"$(cmd)"` plutôt que `"`cmd`"`

- "\${var:-defaultValue}" pour définir une valeur par défaut de la variable var

Shellcheck pour vérifier les erreurs courantes: <https://github.com/koalaman/shellcheck>

Ressources

- [Bash Pitfalls](#)
- [5 Bash String Manipulation Methods That Help Every Developer](#)
- [comm: working with sets and multisets at the command line](#)
- [The TTY demystified](#)
- [grep Flags – The Good Stuff – zwischenzugs](#)
- [12 Useful 'sed' Commands In Linux | LinuxTeck](#)
- [How to write idempotent Bash scripts](#)
- [An Opinionated Guide to xargs](#)
- [Bash Function Names Can Be Almost Anything](#)
- [Linux Commands - A practical reference](#)
- [BashFAQ - Greg's Wiki](#)
- [GNU Coreutils Cheat Sheet](#)
- [Decoded: GNU coreutils – MaiZure's Projects](#)
- [Bash scripting cheatsheet](#)

Tools:

- Linter Bash:
 - <https://github.com/koalaman/shellcheck>
- CLi / visuelle
 - <https://github.com/sharkdp/bat>
 - <https://github.com/lsd-rs/lsd>
 - <https://github.com/ogham/exa>
- Alternatives à find:
 - <https://github.com/tavianator/bfs>
 - <https://github.com/sharkdp/fd>
- Alternative à ctrl+r :
 - <https://github.com/junegunn/fzf> (il fait bien plus de chose aller voir!)
 - <https://github.com/cantino/mcfly>

- Alternative à `man` :
 - <https://github.com/tldr-pages/tldr>
 - <https://github.com/chubin/cheat.sh>
- Manipulation de JSON:
 - <https://github.com/tomnomnom/gron>
 - <https://github.com/stedolan/jq>